

CS201 Completion

22 January 2024

Olcay Taner YILDIZ

Abstract—Write only ONE function per question. Do not check the input, assume that the input is correct. Unless noted, (i) the input data structure is not empty, (ii) time complexity of the algorithm does not matter, (iii) you can not use extra data structures, (iv) you can use any function given in the data structure.

I. QUESTION (ALGORITHM ANALYSIS)

```

1 int algorithm1(int N){
2   int i, j, k, sum = 0;
3   for (i = 1; i <= N; i++)
4     for (j = i; j <= N - i; j++)
5       for (k = 1; k <= j - i; k++)
6         sum++;
7   return sum;
8 }
```

What does the function `algorithm1` return in terms of N ? Assume that N is even. Show your work.

II. QUESTION (GRAPH)

Modify the breadth first search **linked list** implementation such that it will store the indexes of the nodes of the shortest paths from the start node in *paths* parameter.

```

void shortest(int** paths,
             bool* visited,
             int start)
```

At the end of the execution, the indexes in the `paths[i]` array will show the path visited from node *start* to node *i*. You may assume that the visited array is initialized to false and paths array is already allocated.

III. QUESTION (LINKED LIST)

Write the algorithm

```
LinkedList primeDivisors(int N)
```

in the **LinkedList** class which works as follows:

- Creates a temporary linked list *primes*, which stores the prime numbers until N .
- Using *primes* returns all prime divisors (with repeating) of N .

Let say $N = 200$, the function will return 2, 2, 2, 5, 5. You are not allowed to use any array in the function.

IV. QUESTION (TREE)

Write a recursive method

```
void pathList(LinkedList l)
```

in the **TreeNode** class, which returns the keys on the path in the linked list *l*, where the path is defined by the current parent as follows: If the parent is odd, go left; otherwise go right. Assume that the function is called with an empty linked list for the root node.

V. QUESTION (SORTING)

Suppose arrays representing sets A and B are both sorted. Write a method with **linear complexity** in **MergeSort** class that finds if C is the intersection of sets A and B . You are only allowed to use 1 loop.

```

bool isIntersection(int* A, int* B,
                   int* C,
                   int sizeA, int sizeB,
                   int sizeC)
```

VI. QUESTION (SORTING)

Modify the original insertion sort so that

```
void insertionSort(LinkedList l)
```

it will use the same algorithm but sorts the elements in the linked list *l*. You can only use `getPrevious` method as an external method except getters and setters.